

Agentic AI Tech Stack



Foundations and Design of Agentic AI Systems



Engage and Think



Your team has just launched an AI-powered assistant in your product.

In the first week:

- Some users say it works great.
- Others complain it's slow, sometimes gives incomplete answers, and once even gave outdated information.

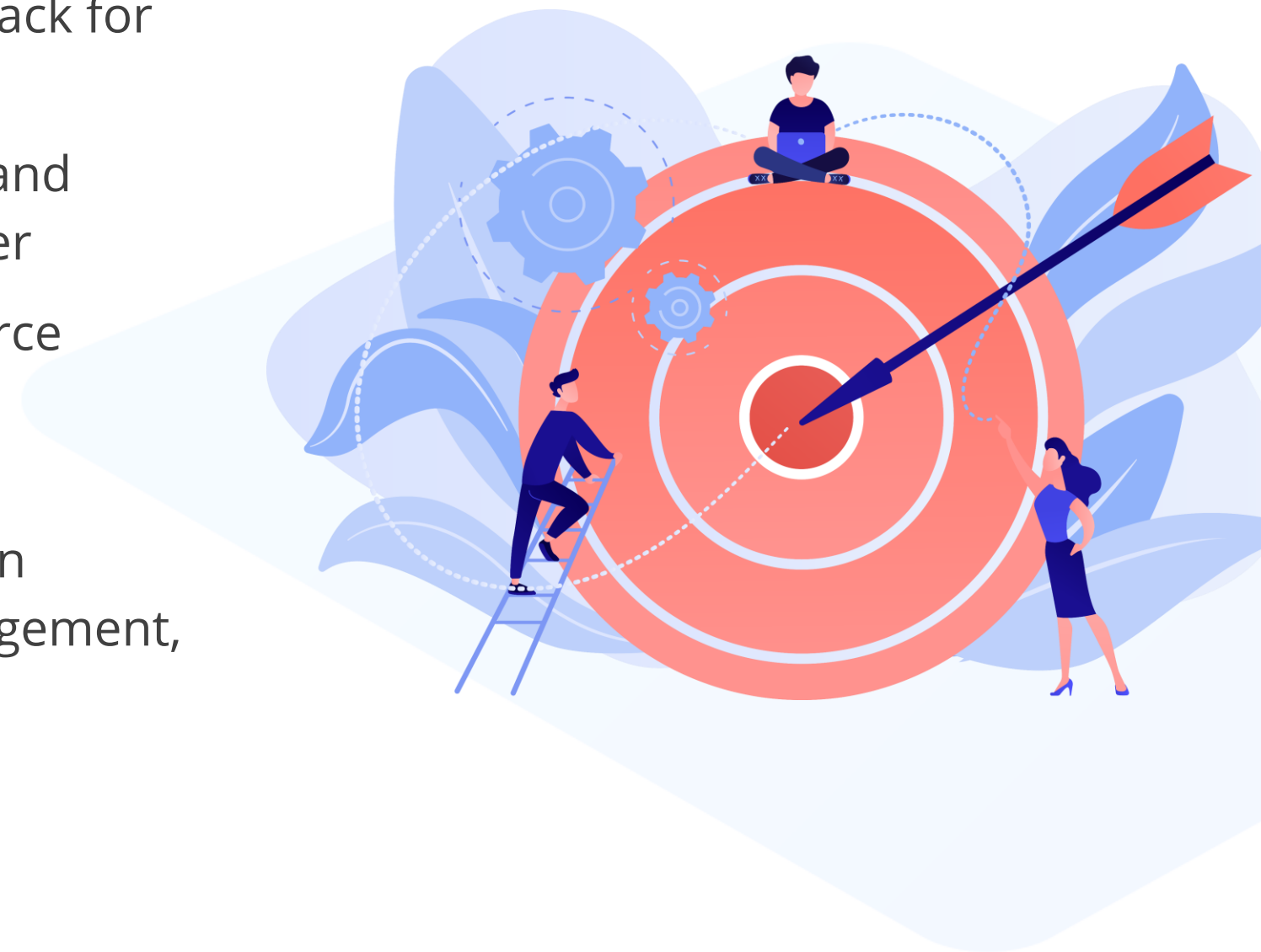
You realize that the tech stack decisions made earlier are now directly affecting the product's performance and user trust.

What stack-level questions would you raise with your tech team to uncover the root cause of these issues?

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the strategic importance of the Agentic AI tech stack for product managers
- Break down the roles of the infrastructure and model layers and define the product manager's key responsibilities at each layer
- Evaluate the trade-offs between closed-source and open-source models, and recommend appropriate strategies for model customization and knowledge access
- Identify and analyze the foundational decisions for building an Agentic AI product, including deployment models, data management, and security considerations





Why the Stack Matters to Product Leaders

Importance of Stack for Product Leaders

Traditional product development focuses on predefined features, whereas agentic systems enable emergent, adaptive product behavior.



Unlike static applications, agentic products act through chained decisions, requiring product leaders to understand how the technical stack shapes their capabilities, limits, and reliability.

The product foundation evolves from merely defining features to orchestrating complex, intelligent behaviors that respond to changing conditions.

Product Behavior: Stack Decisions Drive Outcomes

Agent behavior emerges from choices made across all stack layers, from perception to decision-making.

An agent's ability to parse prompts, retrieve relevant context, and execute multi-step plans depends on tight coordination across components like the model, memory, and tool-calling logic.

Poor choices at any level can lead to hallucinations, failed tasks, or unexpected behavior, which reduces the product's overall value.

Product Behavior: Stack Decisions Drive Outcomes

For example:

Legal reasoning agent

- If chosen vector database is poorly indexed —————> Leads to hallucination
- If RAG system struggles with semantic similarity —————> Leads to irrelevant retrieval of legal precedents

Product managers should understand that the user's experience is shaped by the entire system's design, not just the application itself.

Stack Choices: Causes and Fixes

Surface-level failures in customer service or design often come from deeper stack choices and can be fixed with targeted improvements.

AI customer service agent

If it hallucinates or gives incorrect answers, then it reflects a weak model, poor retrieval, or an underspecified prompt.

To improve:

- Strengthen retrieval pipelines
- Fine-tune the model
- Refine prompt design

Generative design agent

If it produces impossible structures, then it suffers from limited physical reasoning or insufficient simulation resources.

To improve:

- Enhance physics-based constraints
- Expand simulation capabilities
- Validate outputs against real-world parameters

In both cases, surface-level failure traces back to deeper stack choices.

Stack Failures: Major Impact on User Trust

Failures in an agent's core technology, like incorrect answers or security vulnerabilities, can erode user trust and confidence.

Unlike traditional bugs, these failures often stem from drifting model behavior or hidden system dependencies, making them hard to trace.

Teams must identify failure risks across infrastructure, model, and application layers to build robust systems and recovery plans.

A stable and predictable system design is crucial for a reliable product and user confidence.

Stack Failures: Major Impact on User Trust

Examples of stack failures that break user trust are:

AI chatbot generating inappropriate content

- Chatbots like Microsoft Tay created offensive responses because of bad prompts or weak safety settings.
- These failures reflected flaws in model configuration and safety controls, both rooted in stack decisions.

Financial agent providing incorrect advice

- A financial agent may give wrong investment advice due to old information, model mistakes, or corrupted data.
- This failure harmed users and made them lose trust, showing bigger problems within the system's core design.



Understanding the Agentic AI Stack

The Four-Layer Stack: A Framework for Agentic Product Development

Infrastructure layer

This layer provides the fundamental computing, storage, and networking needed for agents to run and data to flow.

1

Model layer

This layer includes the large language models and other AI models that give the system its core intelligence and reasoning.

2

Application/UX layer

This layer is the user-facing interface and product experience, where agent capabilities are used and managed.

4

Orchestration layer

This layer manages the system's multi-step thinking, tool use, memory, and agency, coordinating how models and tools work together.

3

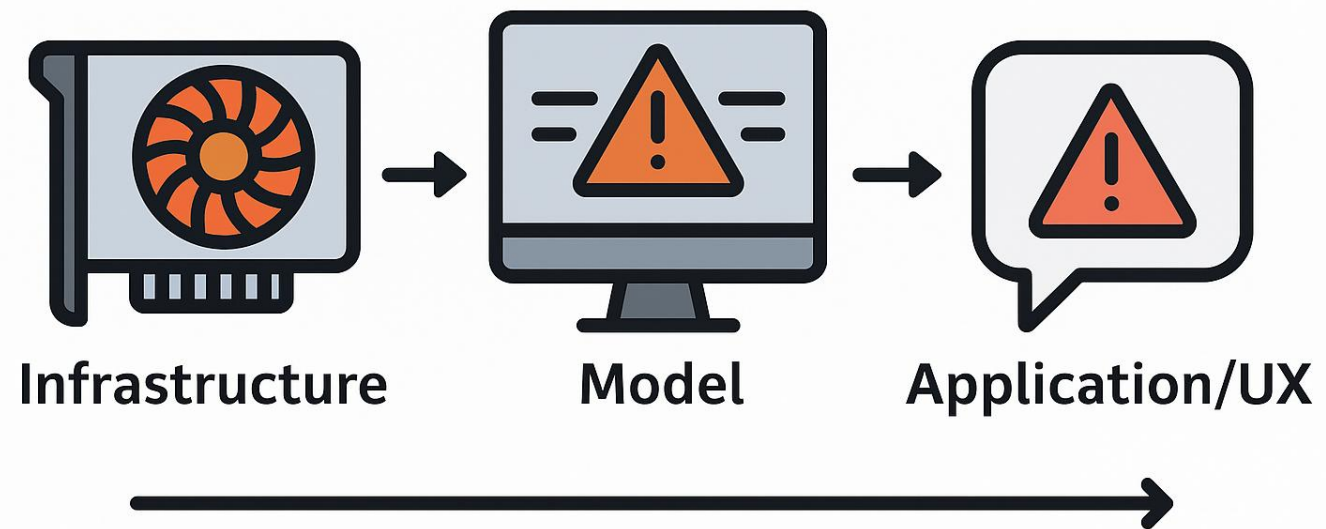
System-Level Thinking for AI Product Leaders

Leading agentic product development requires shifting from focusing on individual features to a comprehensive, system-level view that encompasses the entire technology stack.

- Effective product leaders must understand how all parts of the system interact, how data moves, and how new behaviors emerge.
- Decisions about infrastructure or model adjustments should be judged by their impact on the product's overall performance, cost, and user experience.
- A system-level mindset helps teams proactively find bottlenecks, potential weak points, and innovation chances across the whole agentic design.

System-Level Thinking for AI Product Leaders

GPU-Based Moderation Agent Performance



- For a large content moderation agent, the choice of GPU (Infrastructure layer) affects its speed.
- This speed impacts how quickly harmful content is detected (Model or orchestration layer).
- Faster detection then improves the moderation queue and user safety (Application/UX layer).
- Delays at the infrastructure level directly affect both user experience and business results.

Real-World Analogy: Designing a Legal Reasoning Agent

Companies like *LexisNexis*, *Thomson Reuters*, and various legal tech startups are actively building or exploring such agents.

Infrastructure:

- A highly secure cloud environment like AWS GovCloud is used to meet legal compliance.
- Specialized GPUs and optimized vector databases index vast legal corpora for large legal LLMs.

Model:

- Open-source models like Llama or Mistral might be fine-tuned on proprietary legal datasets.
- Alternatively, highly specialized legal LLMs can be licensed from third parties.

Real-World Analogy: Designing a Legal Reasoning Agent

Orchestration:

- Orchestration tools include legal research APIs (for example: Westlaw, LexisNexis) and internal document management systems.
- Sophisticated planning agents chain tasks such as identifying relevant cases, summarizing key arguments, and drafting motions.

Application/UX:

- The interface allows lawyers to upload documents, pose complex legal questions, and review AI-generated drafts with citations.
- Lawyers can also iterate on the output within their existing workflows.

Quick Check



Your team rolls out an AI support agent, but user feedback reveals accuracy problems. Investigation points to a weak retrieval configuration and an underspecified prompt. What broader insight does this provide about agentic systems?

- A. These issues only occur with open-source models.
- B. Only UX design controls user experience.
- C. Stack-level decisions shape agent behavior and reliability.
- D. Prompt length is the primary driver of performance.



Overview of Infrastructure Layer

Introduction to Infrastructure Layer

The performance and reliability of any agentic product fundamentally depend on its underlying compute and inference infrastructure.

Compute

It is the processing power used for training, fine-tuning, and data processing within the agent workflow, which typically runs offline and leverages GPUs for parallel processing.

Inference

It is the real-time execution of AI models that generate responses or perform actions whenever the agent interacts with a user.

Cloud and GPU Providers

Selecting the right cloud and GPU providers is a pivotal strategic decision, impacting an agentic product's scalability, cost, and access to specialized hardware.



Major cloud providers

AWS, Azure, and GCP provide powerful AI tools and GPU-based servers. Some, like Google and AWS, also design their own AI chips such as TPUs and Inferentia.

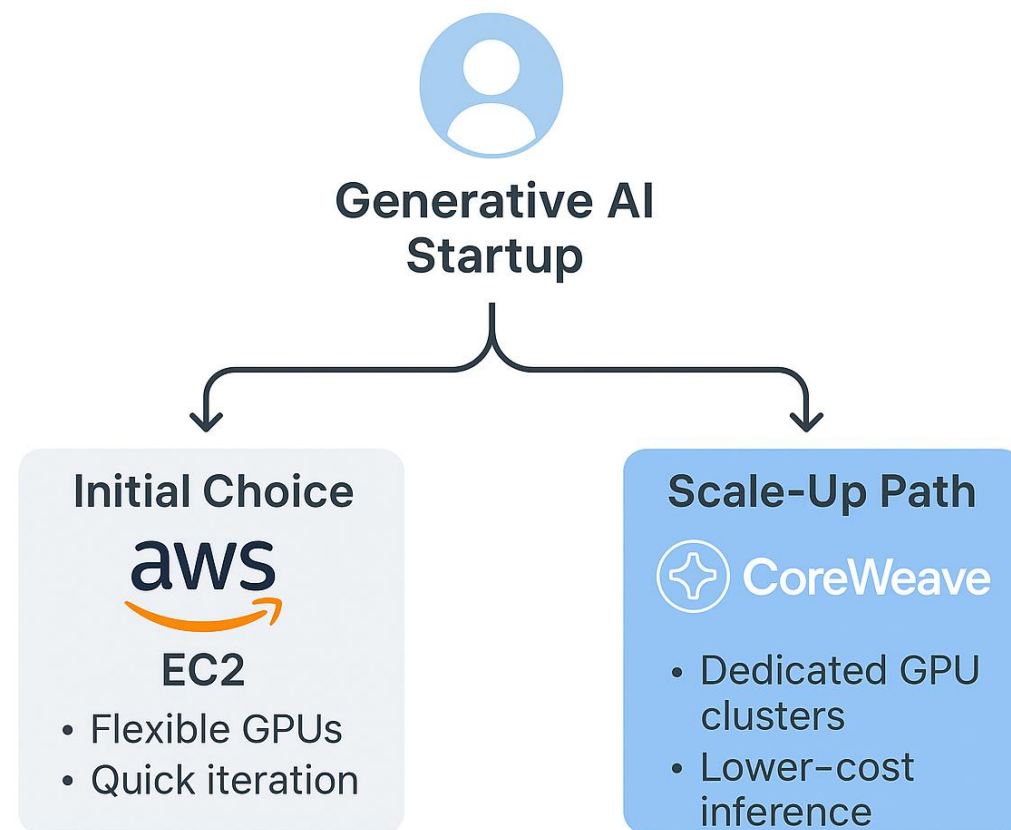


Specialized providers

CoreWeave or Lambda Labs focus on GPU infrastructure, offering high-performance, cost-effective solutions for large-scale AI training and inference.

Cloud and GPU Providers

Product leaders must assess providers based on instance types, regional coverage, pricing models, and AI-specific services that match product needs.



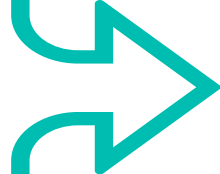
Example: Generative AI art product

- A startup creating a generative AI art tool may start with AWS EC2 using NVIDIA A100 GPUs for flexibility and scale.
- As demand grows, they may shift to CoreWeave for lower-cost, dedicated GPU clusters optimized for inference at scale.

Self-Hosting vs. Cloud Deployment

Choosing between self-hosting and cloud services requires balancing control and convenience, with strategic implications.

- Self-hosting gives full control over hardware, data, and customization, which is critical for regulated industries where data sovereignty and security are essential.
- It reduces long-term inference costs at scale but requires high upfront investment.

- 
- A financial institution may self-host its fraud detection agent to retain full control over data security and sovereignty.
 - This model favors privacy over cost, despite higher initial investment.

Self-Hosting vs. Cloud Deployment

Cloud deployment provides scalability, managed services, and lower initial costs, enabling faster iteration but possibly increasing long-term expenses.



Example

A chatbot startup would likely choose cloud deployment (for example, AWS, Azure, or GCP) to scale quickly, reach global users, and leverage managed services, deferring major capital costs.

Product leaders must weigh these trade-offs against needs like data sensitivity, compliance, performance, and operational maturity.

Hybrid and Edge Deployment Use Cases for Agentic Products

Hybrid and edge models combine cloud power with local execution to cut latency and boost responsiveness.

Hybrid deployment

It integrates on-premises and cloud to keep sensitive data local and scale for bursts or training.

Edge deployment

It places inference near users or data (devices, local servers) to enable ultra-low latency, offline use, and privacy.

Product managers should adopt hybrid or edge strategies where latency, sovereignty, or connectivity challenges are critical.

Vendor Lock-In and Portability Trade-Offs

Strategic infrastructure choices must weigh vendor lock-in risks against the benefits of portability.

- Using proprietary AI services (for example, custom APIs, managed databases) can boost performance but may limit migration flexibility.
- Open-source tools and containers (for example, Docker, Kubernetes) increase portability but may add complexity or require specialized skills.

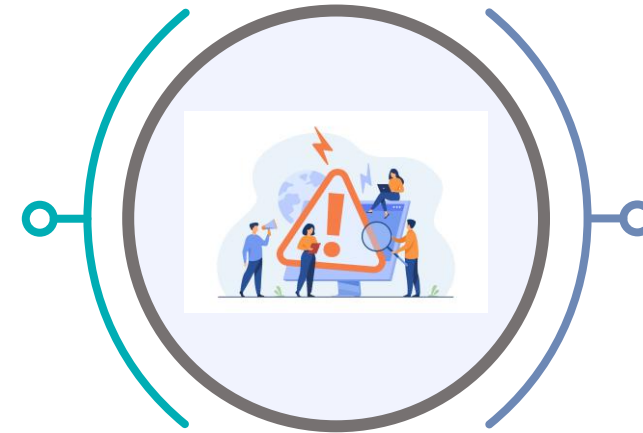
Example:

- A company using OpenAI APIs or Azure Cognitive Services may move faster but risks rework if switching providers later.
- Adopting Hugging Face and Docker or Kubernetes improves portability but demands deeper internal MLOps expertise.

Autoscaling Challenges in Agentic Systems

Autoscaling in agentic systems is challenging because inference demand can spike suddenly, and AI models require specialized resources like GPUs.

- Efficient autoscaling requires monitoring more than CPU or memory.
- This includes key metrics like GPU utilization, token throughput, and concurrent inference requests to AI models.



- Over-provisioning wastes money, while under-provisioning causes latency, queuing, and degraded user experience.
- This directly impacts the product reliability.

Example:

A shopping agent preloads GPUs before Black Friday to avoid slowdowns.

Example:

A fraud agent keeps baseline GPUs to handle sudden spikes efficiently.

Autoscaling Challenges in Agentic Systems: Example

A holiday shopping assistant agent experiences massive spikes in usage during Black Friday.



- Traditional CPU-based autoscaling often fails to handle sudden GPU demand from LLMs.
- Without tuning for token throughput or concurrent requests, systems may slow down, frustrating users.

Latency Zones and Regional Model Access

Geographic distribution of users and the physical location of model inference endpoints significantly impact an agentic product's latency and responsiveness.

Example: Global customer support agent

- A global customer support agent should have inference endpoints in Asia, Europe, and North America.
- If all traffic goes through a single U.S. data center, users in Asia may face high latency. This slows response times and worsens the user experience.
- Regional deployment also helps meet data residency laws, such as GDPR in Europe.



Introduction to Monitoring and LLMOps

LLMOps: Monitoring and Observability for Agentic Systems

LLMOps (Large language model operations) extends DevOps principles to the lifecycle of LLM-powered agentic systems, with monitoring and observability as core tenets.

Monitoring: Tracks model performance (accuracy, latency), infrastructure, and multi-step agent behavior

Observability: Offers deep insight into an agent's internal states, which is crucial for debugging and continuous improvement

Result: Robust LLMOps practices ensure proactive problem detection, performance optimization, and responsible AI governance.

Model Behavior: Rollbacks and Drift Detection

Agentic systems require robust strategies for model rollback and drift monitoring to ensure stable behavior in production.

Model rollbacks

Allows quick rollback to a previously stable model version or configuration when performance degrades or the agent behaves unpredictably

Drift detection

Tracks discrepancies between expected and actual output to detect issues caused by shifting user inputs (concept drift) or data sources (data drift)

Model Behavior: Rollbacks and Drift Detection

Implementing automated drift detection and rapid rollback capabilities is paramount for maintaining the reliability and trustworthiness of agentic products in a constantly evolving environment.

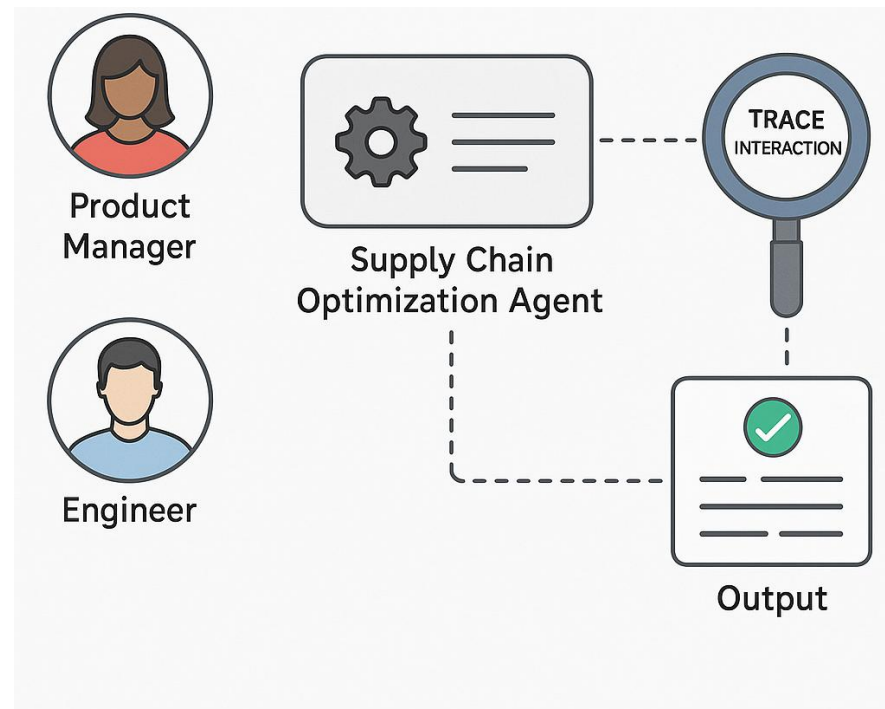


Example: AI-powered content moderation agent

- An AI content moderation agent may begin flagging valid content after a model update.
- Drift detection techniques, such as monitoring flagged content patterns or cosine similarity of embeddings over time, can alert the team to issues.
- This enables a fast rollback to a stable version, minimizing false positives.

Token-Level Debugging Tools for Granular Agent Analysis

Product managers, in collaboration with engineers, can leverage these tools to rapidly identify root causes of unexpected agent behavior, leading to faster iteration and improved product quality.



- If a supply chain optimization agent is making suboptimal decisions, a product manager and engineer might use tools like **LangChain's LangSmith** or **OpenAI's playground logs** to trace a specific interaction.
- They might discover that the agent correctly identified a relevant data point but misinterpreted a date from a tool's output.
- This mistake during its reasoning process could result in a flawed final recommendation.



Introduction to Data Storage

Data Storage Solutions for Agentic Products

Effective agentic products rely on robust and varied data storage solutions to manage the diverse types of data they process, from raw inputs to model outputs and persistent memory.

Structured databases

(SQL/NoSQL), PostgreSQL, MongoDB, and Cassandra

Object storage

Amazon S3, Google Cloud Storage, Azure Blob Storage

Vector databases

Pinecone, Milvus, Qdrant, Weaviate

Data Storage Solutions for Agentic Products

These solutions must support high throughput, low latency, and scalability to handle the demands of dynamic agent interactions.

Structured databases

Ideal for managing agent configurations, user profiles, transactional data, and structured memory

Object storage

Ideal for large volumes of unstructured data like raw documents, images, audio files, and model checkpoints

Vector databases

Specifically designed for storing and querying high-dimensional embeddings, crucial for retrieval-augmented generation (RAG) and semantic search

Data Storage Solutions for Agentic Products

Product managers must understand the trade-offs between these storage types in terms of cost, performance, data consistency, and suitability for different agentic use cases.

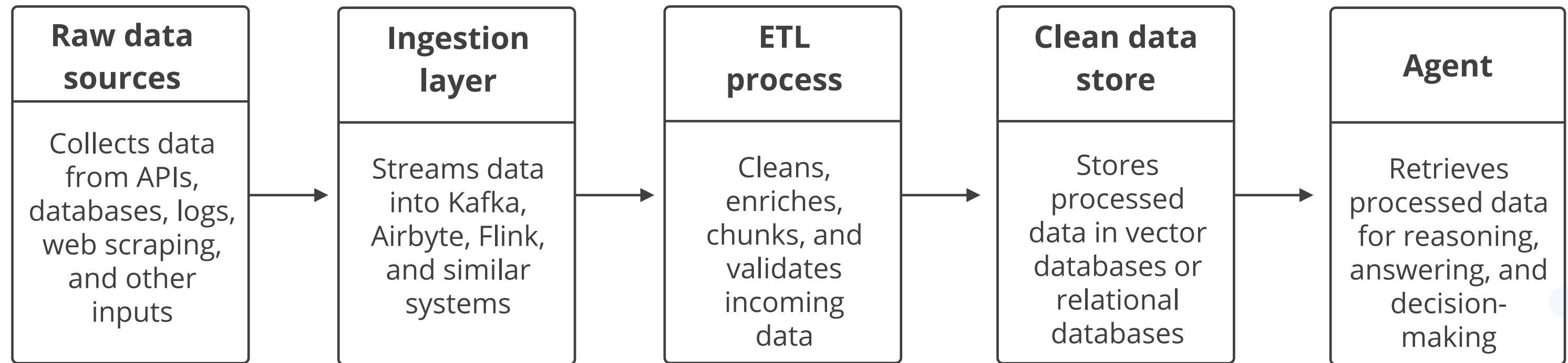


- A content creation agent might use Amazon S3 to store large corpora of text and images for training or fine-tuning.
- It may use a PostgreSQL database for structured data such as user preferences and content metadata.
- It may also use Pinecone to store embeddings of generated content for semantic search and versioning.

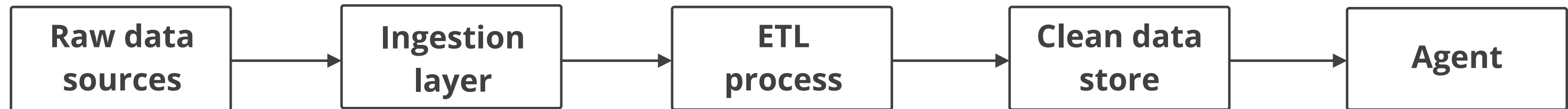
Data Ingestion and ETL for Agentic Workflows

Efficient data ingestion and Extract, Transform, Load (ETL) pipelines are vital for feeding high-quality, up-to-date information to agentic systems, especially for RAG and continuous learning.

These pipelines ensure that raw data is collected, cleaned, transformed, and loaded into appropriate storage solutions in a timely and reliable manner.



Data Ingestion and ETL for Agentic Workflows



Real-time vs. batch processing:

Agents often require near real-time data updates for current information, while historical data might be processed in batches.

Data connectors:

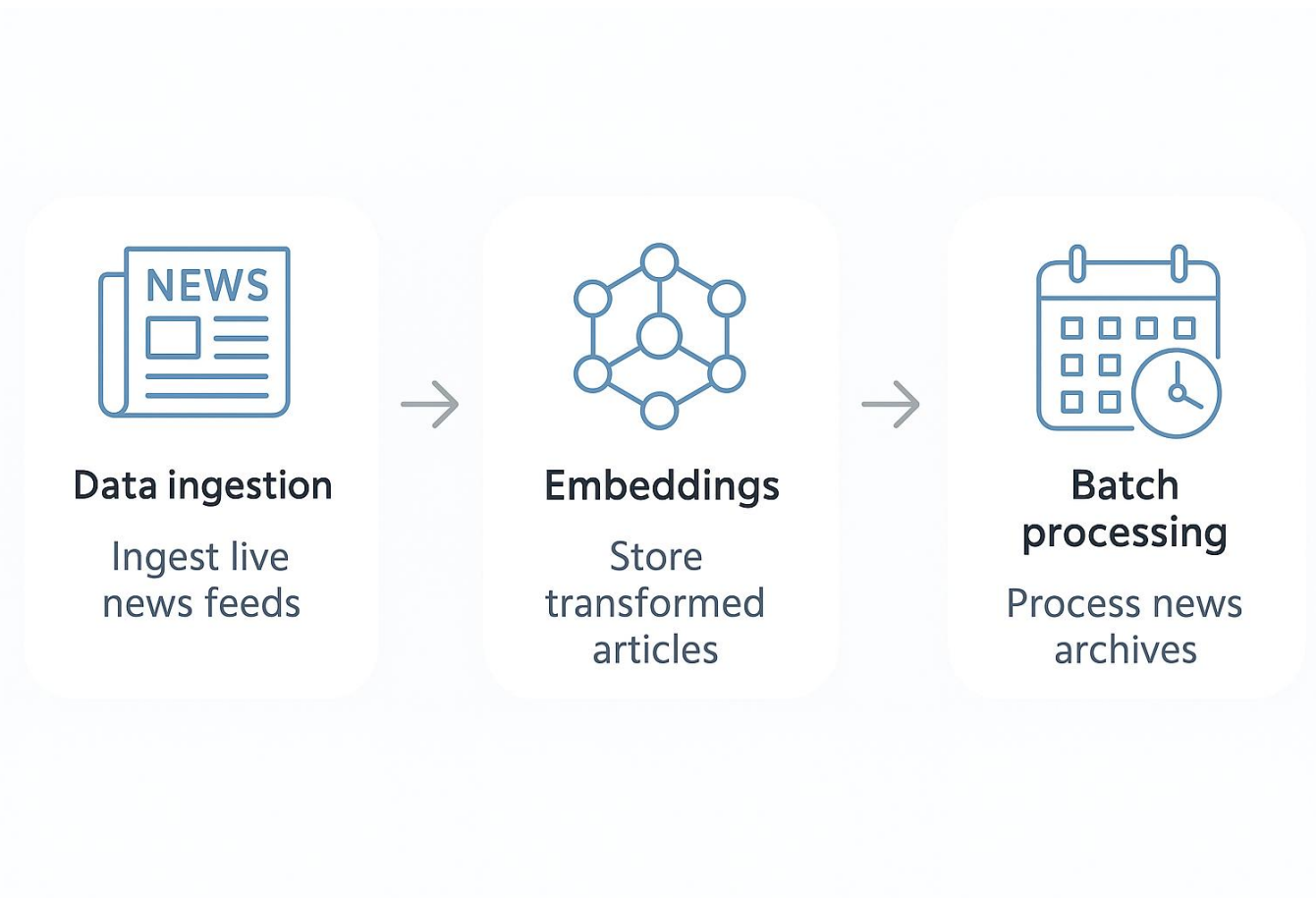
Integration with various data sources such as APIs, databases, and streaming platforms like Kafka is essential.

Data transformation:

Cleaning, normalizing, enriching, and chunking data (for RAG) are critical steps to prepare it for model consumption.

Data Ingestion and ETL for Agentic Workflows

Product leaders must advocate for robust data pipelines that guarantee data freshness, integrity, and accessibility, as these directly impact agent accuracy and responsiveness.



- A news analysis agent might use a real-time data ingestion pipeline (for example, Apache Kafka or AWS Kinesis) to ingest live news feeds.
- It transforms incoming articles into embeddings and stores them in a vector database for immediate retrieval-augmented generation (RAG).
- Historical news archives may be processed daily through a batch ETL job for long-term analysis or enrichment.

Data Governance and Compliance for Agentic AI

Data governance and compliance are paramount for agentic products, especially when dealing with sensitive or regulated information. Product managers must ensure that the following data handling practices adhere to legal, ethical, and organizational standards:

Data privacy regulations

Compliance with GDPR, CCPA, HIPAA, and other privacy laws is essential for agents handling PII or health data.

Data residency and sovereignty

Ensuring data is stored and processed within specific geographical boundaries as mandated by law or policy.

Data retention policies

Defining how long data is stored and when it must be purged.

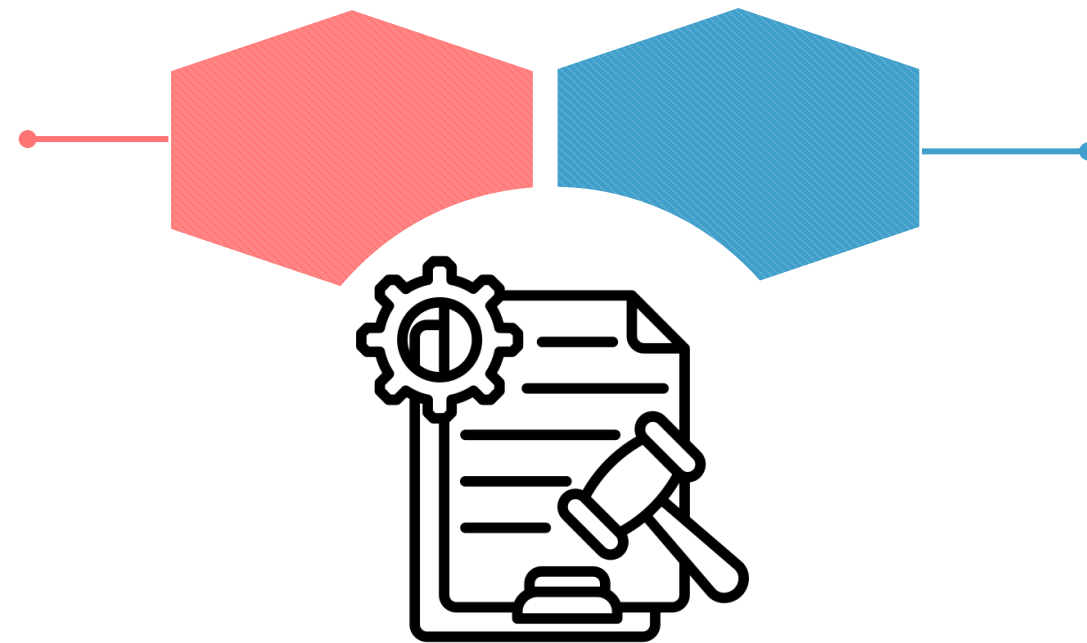
Consent management

Establishing clear processes for obtaining and managing user consent for data collection and use.

Data Governance and Compliance for Agentic AI

Example: A healthcare assistant agent that processes patient data must be designed with **HIPAA compliance** in mind.

This means ensuring data encryption at rest and in transit, strict access controls, auditable logs of data access, and secure data deletion policies.



Poor data governance can lead to severe legal penalties, reputational damage, and erosion of user trust.

The product manager must work closely with legal and compliance teams to define these requirements.

Data Versioning and Lineage

Maintaining data versioning and lineage is crucial for reproducibility, debugging, and auditability of agentic systems, particularly for models that are continuously trained or fine-tuned.

Data versioning:

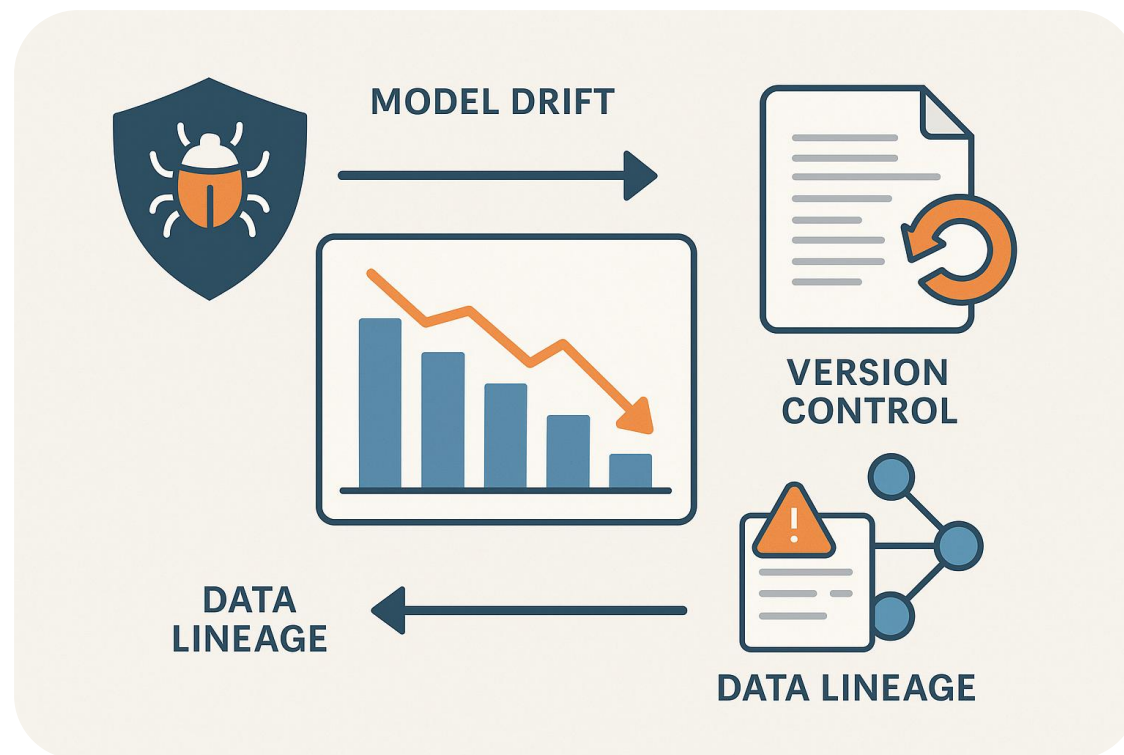
Tracking changes to datasets over time, allowing for recreation of specific model training runs or agent behaviors.

Data lineage:

Documenting the origin, transformations, and destinations of data as it flows through the agentic pipeline, providing transparency and accountability.

Data Versioning and Lineage

Product leaders should ensure that MLOps practices include robust data versioning and lineage tools, as these are foundational for model reliability and explainability.



- **Example:** For a fraud detection agent, if a new model version shows a drop in performance, data versioning tools (for example, **DVC - Data Version Control**) allow the team to revert to the exact dataset used for the previous, well-performing model.
- Data lineage helps trace whether a new data source or transformation step introduced unexpected errors.

Data Quality and Cleansing for Agentic Performance

Clean, high-quality data is key to accurate models, effective RAG, and reliable agents. The following steps help ensure accuracy, efficiency, and trust in AI outputs:

- **Data validation**

Implementing checks to ensure data conforms to expected formats, ranges, and types

- **Duplicate detection**

Identifying and merging or removing redundant data entries

- **Missing value imputation**

Handling missing data using techniques like averaging, similarity-based filling, or predictive modeling

- **Outlier detection**

Identifying and managing anomalous data points that could negatively impact model training or inference

Data Quality and Cleansing for Agentic Performance

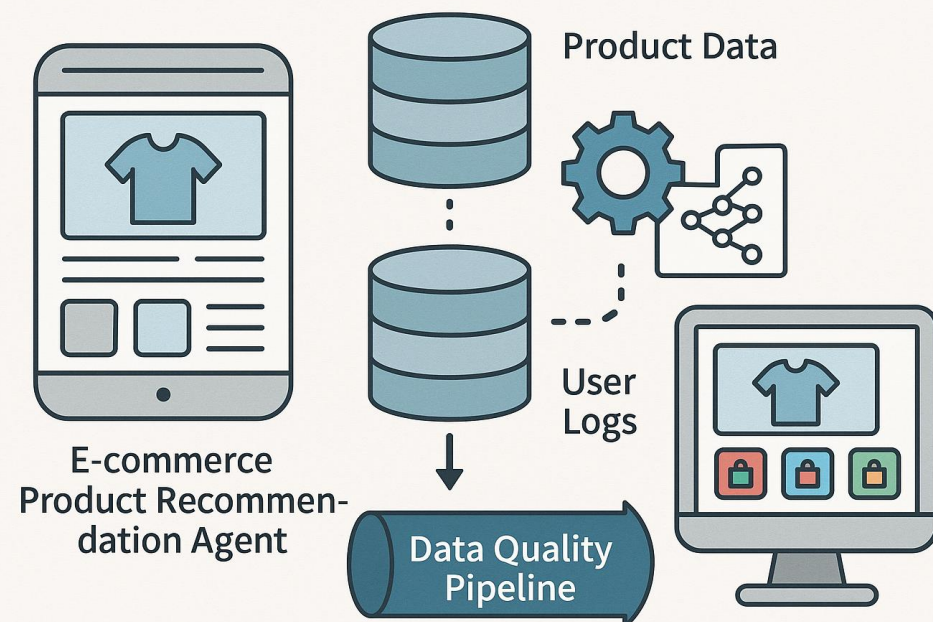
Product managers must prioritize data quality initiatives, understanding that investments in data cleansing directly translate to improved agent intelligence and reduced hallucinations.

Example:

An e-commerce product recommendation agent relies on product catalog data and user interaction logs.

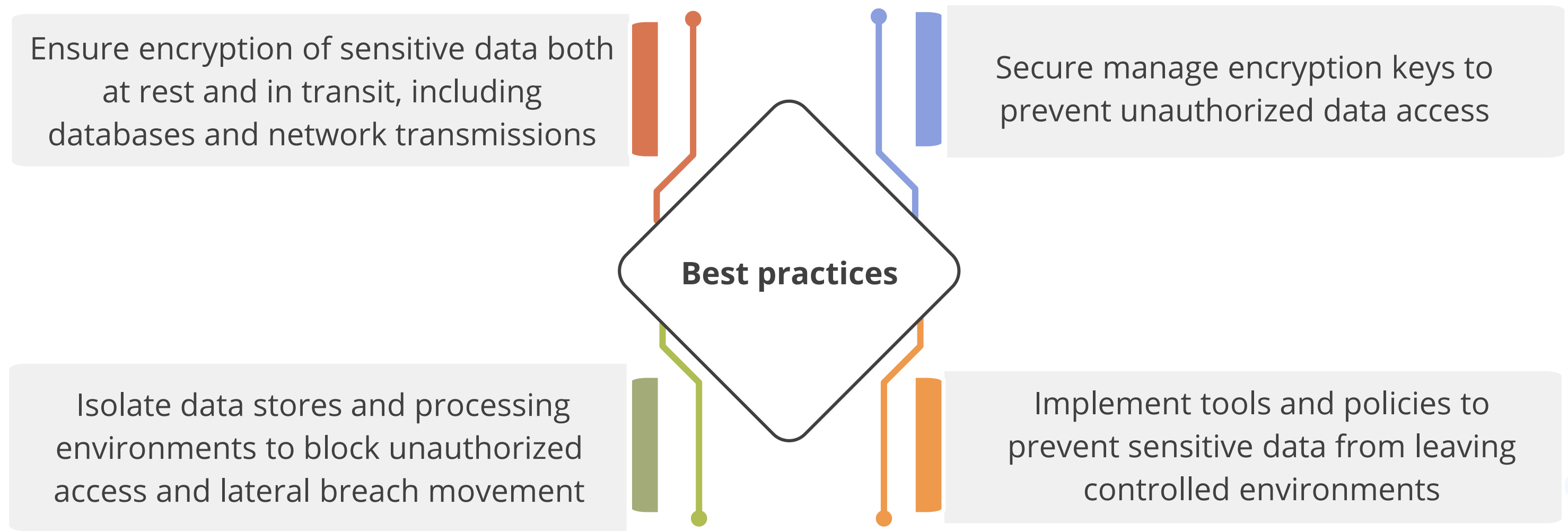
If product descriptions contain inconsistent spellings, missing attributes, or duplicate entries, the agent's recommendations will be poor.

Data quality pipelines must cleanse and standardize this information to ensure accurate and relevant suggestions.



Data Security in Agentic Infrastructure

Beyond general network and system security, data security in the infrastructure layer specifically focuses on protecting the data itself, whether at rest, in transit, or during processing.



Access Control and Authentication for Agentic Systems

Robust access control and authentication mechanisms are fundamental to limiting unauthorized access to agent components, data, and functionalities.

Role-based access control (RBAC):

Defining specific roles (for example, admin, engineer, end-user) with granular permissions to different parts of the agentic system and its underlying data

Multi-factor authentication (MFA):

Requiring multiple forms of verification for privileged access

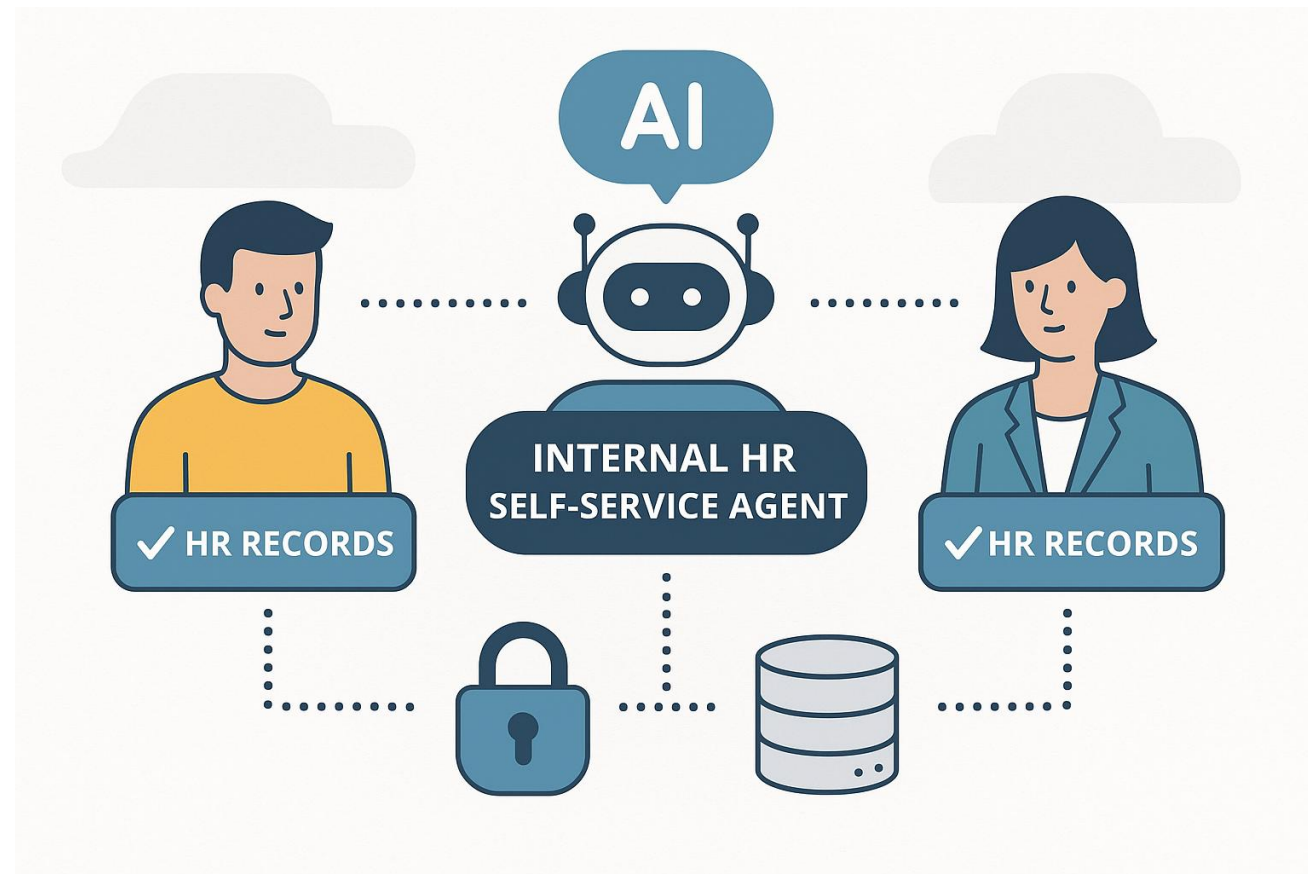
Identity and access management (IAM):

Centralized management of user identities and their access privileges across the entire tech stack



Access Control and Authentication for Agentic Systems

Product managers must define user roles and access requirements early in the design phase to build security from the ground up. This ensures that only authorized entities can interact with sensitive data and agent capabilities.



- An internal HR agent for employee self-service would use RBAC to ensure that employees can only access their own HR records, while HR administrators have broader access.
- The agent itself would have specific service-to-service authentication credentials to securely retrieve data from the HR database.

Audit Trails and Logging for Data Access

Comprehensive audit trails and granular logging of all data access and modifications within the agentic infrastructure are critical for security, compliance, and post-incident forensic analysis.

Granular event logging:

Recording details such as who accessed what data, when, from where, and what actions were performed

Immutable logs:

Ensuring that logs cannot be tampered with or deleted after they are generated



Centralized log management:

Aggregating logs from various infrastructure components such as databases, servers, and APIs into a centralized system for easy analysis and alerting

Disaster Recovery and Data Backup Strategies

Agentic products must be designed with resilience in mind, incorporating robust disaster recovery and data backup strategies to ensure business continuity and data availability in the face of outages or data loss.

Regular backups:

Implementing automated, scheduled backups of all critical data stores (databases, object storage, model checkpoints)

Geographic redundancy:

Storing backups and potentially replicating active data across multiple geographic regions to protect against regional disasters

Recovery Point Objective (RPO) and Recovery Time Objective (RTO):

Defining clear targets for how much data can be lost (RPO) and how quickly the system must be restored (RTO) after a disaster

Failover mechanisms:

Automated systems to switch to redundant infrastructure components in case of failure

Quick Check



A health chatbot uses patient records to suggest treatments. After launch, it's flagged for violating HIPAA norms. What was most likely missing in the design?

- A. An improved RAG setup
- B. Multimodal input validation
- C. Consent management and compliance checks
- D. GPU logging transparency



Introduction to Model Layer

LLM Selection: Open vs. Closed, Size, and Cost

Choosing the right LLM impacts reasoning quality, cost, control, and product economics.

Closed-source models

- Offer top-tier performance and robust safety features
- Easy to use via APIs
- Higher per-token cost and vendor lock-in
- Best for rapid prototyping and premium use cases
- **Example:** OpenAI GPT, Claude, and Gemini

Open-source models

- Provide control and customizability via fine-tuning
- Lower costs at scale
- Require infrastructure and in-house expertise
- Suitable for long-term deployment and safety tuning
- **Example:** Llama 3, Mistral, Gemma, and Falcon

Prompt Engineering: Agent Communication and Behavior Control

Prompt engineering is both an art and a science, focused on crafting effective inputs (prompts) to guide an LLM's behavior and elicit desired outputs from an agent.

Techniques: Include clear instructions, few-shot examples (providing a few input/output pairs), role-playing (act as a senior legal analyst), chain-of-thought prompting (asking the agent to think step-by-step), and specific output format requests like JSON and markdown.

Impact: It's a continuous process of refinement based on observed agent behavior.

Prompt Engineering: Agent Communication and Behavior Control

Product managers must define agent personas, refine task framing, and clearly instruct interaction behavior through precise prompts.

Example: Sales support agent empathy

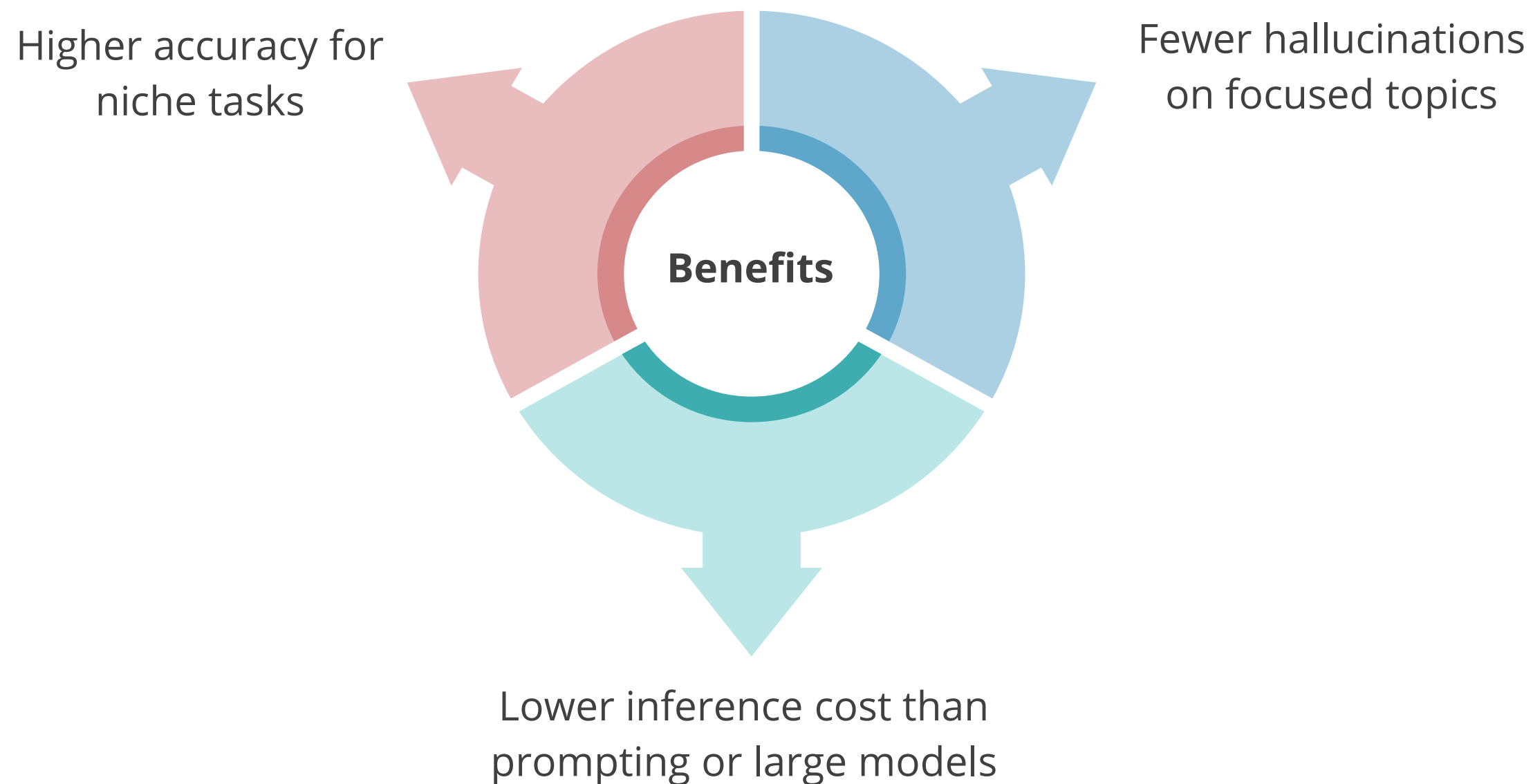
Instruct the agent to act as a friendly, patient sales associate (for example, Adopt the persona... Confirm understanding before proceeding) to guide tone and style

Example: Code review agent output format

Use structured prompts (for example, critique for PEP 8 compliance, output in JSON with line number, severity, description) to control output format and focus

Fine-tuning and Model Adaptation

Unlike prompting, fine-tuning adapts a base model by retraining it on domain-specific data for better specialization and accuracy.



Fine-tuning and Model Adaptation

Product leaders should consider fine-tuning when off-the-shelf models underperform on domain-specific tasks or when critical workflows require consistent, high-quality output.

Methods:

Full fine-tuning:

- Retrains all parameters on a new dataset.
- Is resource-intensive but delivers high specialization

Parameter-efficient fine-tuning (PEFT):

Uses methods like LoRA and QLoRA to fine-tune a small subset of parameters, reducing compute and memory use

Domain adaptation:

Trains the model on large volumes of unlabeled domain-specific text (for example, legal documents, medical journals) to build context-specific vocabulary and reasoning

Embedding Models: Foundation for Semantic Understanding

Embedding models turn text, images, or other data into numerical vectors that capture their meaning, allowing agents to find and use relevant information.

Key roles of embedding models in agentic systems:

Serves as the core of Retrieval-Augmented Generation (RAG)

Returns the most relevant information chunks as context for the LLM

Transforms user queries into embeddings for semantic search over a knowledge base

Determines retrieval accuracy, hallucination risk, and the agent's ability to respond factually

RAG for Context-Aware Responses

It combines an LLM with a retrieval system to ground responses in relevant, external data.

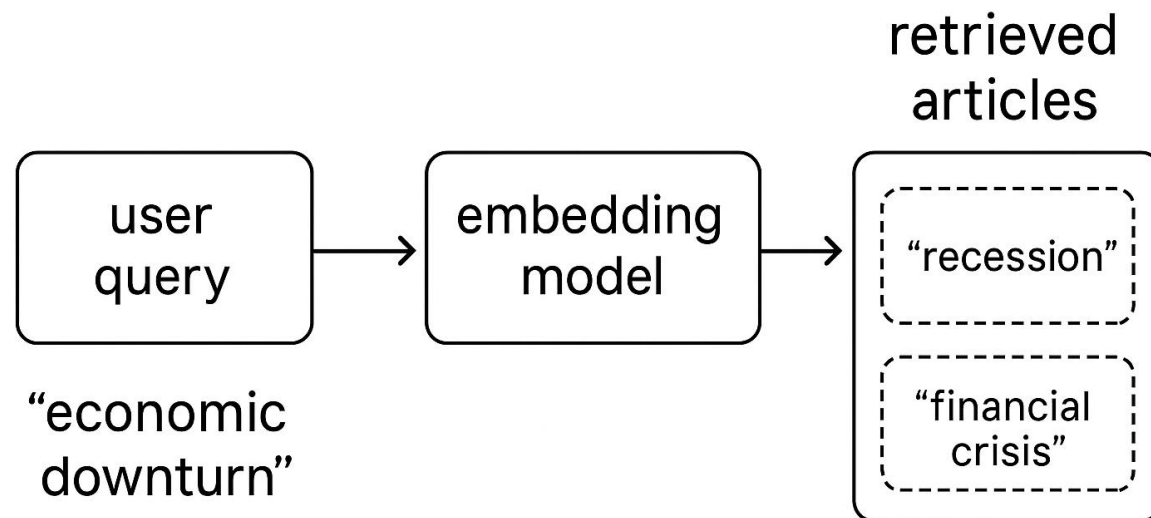
It reduces hallucinations by supplementing the model's reasoning with factual context.

It allows dynamic updates without retraining the base model.

It is ideal for products that require domain-specific, up-to-date information.

Embedding Models: Foundation for Semantic Understanding

Product managers must recognize that embedding quality is crucial for agent performance in data-heavy scenarios. Poor embeddings compromise context relevance and result in weak responses.



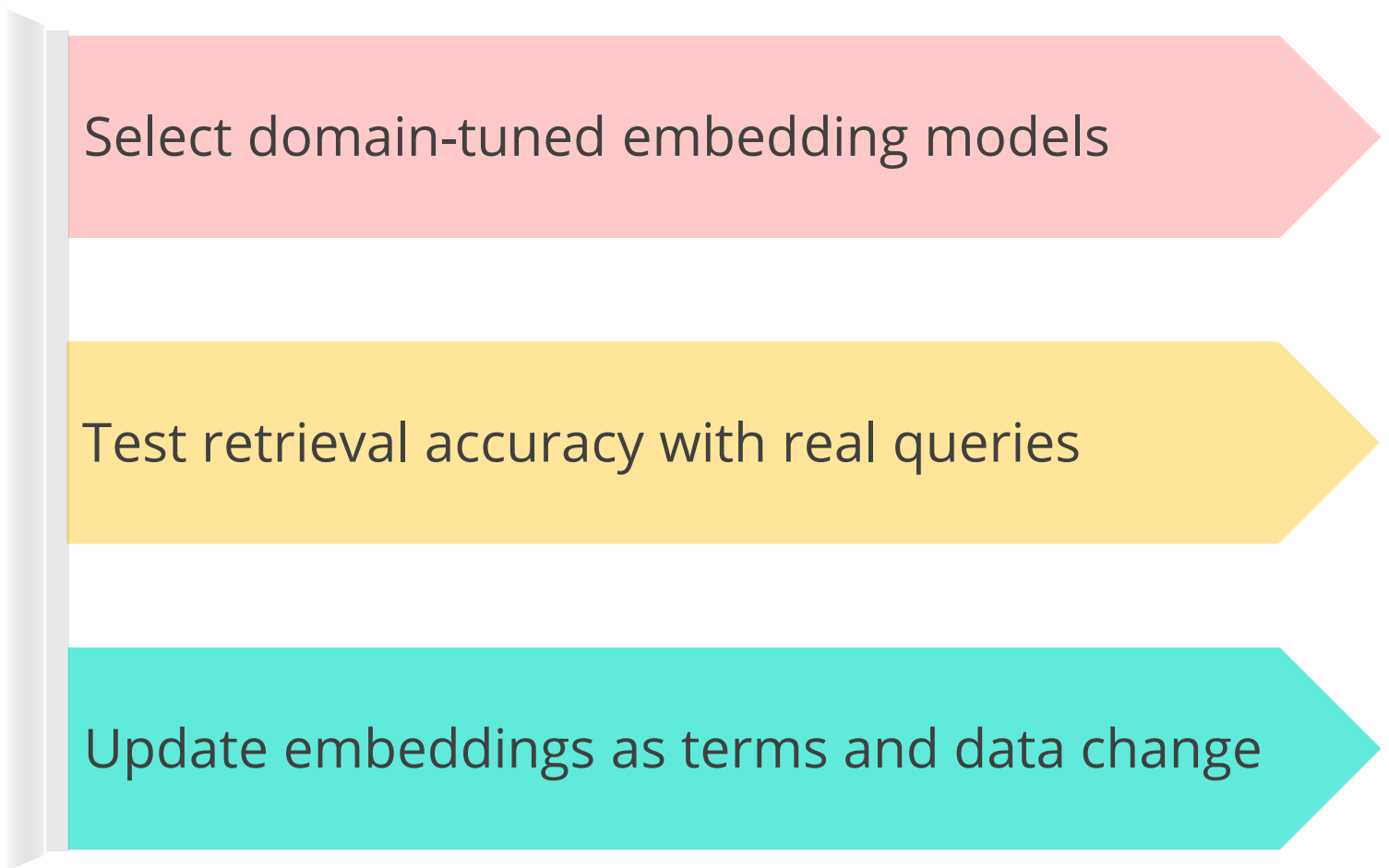
Example: News summarization agent

- Well-tuned embeddings retrieve articles that match the meaning of the query, even if keywords differ.
- A query on economic downturn should return content about recession or financial crisis.

Embedding Models: Foundation for Semantic Understanding

Product managers must recognize that embedding quality is crucial for agent performance in data-heavy scenarios. Poor embeddings compromise context relevance and result in weak responses.

PM actions for better embeddings:



Select domain-tuned embedding models

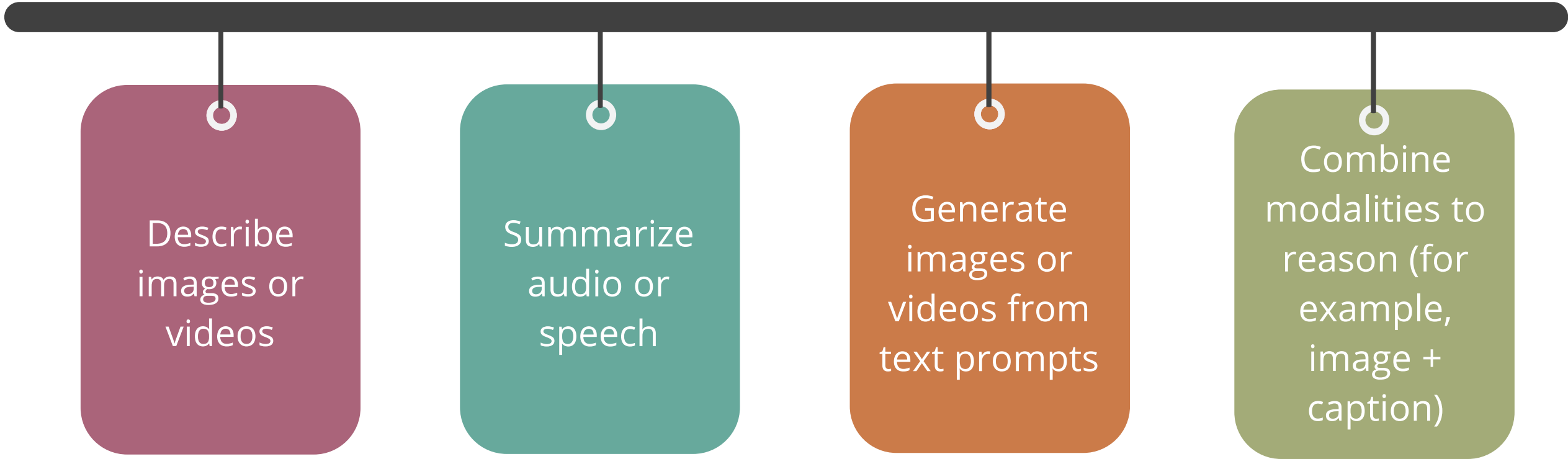
Test retrieval accuracy with real queries

Update embeddings as terms and data change

Multimodal Models for Rich Agent Interactions

Multimodal models extend agent capabilities beyond text by processing and generating content across text, images, audio, and video. This enables agents to see, hear, and understand the world.

Multimodal models can:



Describe
images or
videos

Summarize
audio or
speech

Generate
images or
videos from
text prompts

Combine
modalities to
reason (for
example,
image +
caption)

Multimodal Models for Rich Agent Interactions: Examples

Product leaders can leverage multimodal capabilities to create agents that mirror human perception and interaction.

Interior design agent:

Takes a room image, analyzes layout and furniture, and suggests improvements or generates new visuals (For example: Make this room minimalist.)

Virtual event agent:

Listens to live speeches, transcribes audio, and generates real-time captions or visual summaries to boost accessibility

Model Evaluation

Evaluating models goes beyond accuracy. It reveals the agent's true strengths and limitations.

Key metrics for LLMs and agents:

Factual consistency

Is the output factually correct and context-grounded?

Fluency

Is it coherent, grammatical, and natural-sounding?

Instruction adherence

Does it follow the prompt precisely?

Bias and toxicity

Does it avoid harmful or inappropriate content?

Latency and throughput

How fast and efficiently does it perform?

Tool use

Does it pick the right tool, use it properly, and apply its results correctly?

Model Benchmarking

Benchmarks offer structured ways to assess performance across reasoning, behavior, and tool use.

Popular frameworks are:

MMLU

Tests multi-discipline reasoning

- Provides a % of correct answers across 57 subjects (like social sciences and more)

HELM

Evaluates accuracy, bias, toxicity, and cost

- Provides a multi-metric dashboard with comparative scores

MT-Bench

Uses LLM-as-judge to score instruction-following

- Provides a rating between 1 to 10 based on response quality against prompts

ToolBench

Assesses how well agents use tools

- Provides scores for task success rate and tool execution accuracy

Model Interpretability and Explainability (XAI)

Understanding how and why models make decisions is essential for building trustworthy AI.

Key concepts:

- **Interpretability:** Ability to inspect internal mechanisms or reasoning steps
- **Explainability:** Ability to describe why a model produced a specific output

Why it matters:

- Builds trust with users and stakeholders
- Enables safer deployment in sensitive domains
- Helps debug and improve unpredictable behaviors

Model Bias and Fairness in Agentic Systems

LLMs often reflect societal and data-driven biases. These biases can lead to unfair or offensive outputs if not addressed.

How bias shows up:

- Reinforces stereotypes in generated content
- Produces skewed responses across identity groups
- Causes uneven or harmful user experiences

How to mitigate:

- Fine-tune with more representative data
- Use filters or rewriting layers for sensitive outputs
- Run fairness audits and implement bias metrics during evaluation

Fairness isn't just a technical concern; product teams must own the user impact of model behavior.

Model Bias and Fairness in Agentic Systems

Product leaders are responsible for championing ethical AI principles, working closely with data scientists, ethicists, and legal teams to design and deploy agents that are fair, inclusive, and responsible.

Example: Loan eligibility agent

An agent that denies loans more often to specific groups can reinforce financial bias even without intent.

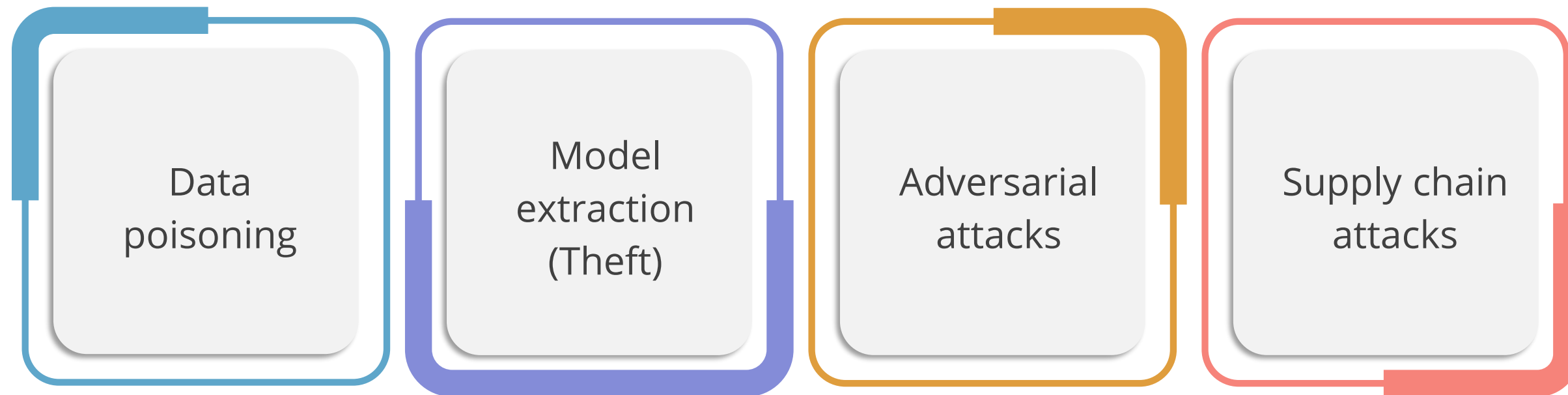
PM responsibilities:

- Monitor model impact on different user groups
- Collaborate across ML, design, and legal teams
- Flag and address bias early in development
- Build transparency and fairness into the product experience

Model Security Beyond Prompt Injection

Agentic systems face deeper security threats beyond prompt injection.

These risks can compromise model performance, intellectual property, and user trust if not addressed in collaboration with security teams, and key threats to watch are:



Key Takeaways

- Agentic systems shift product design from static features to emergent behavior, which makes stack-level decisions central to product outcomes.
- The agentic stack consists of infrastructure, model, orchestration, and application layers, and each one plays a direct role in shaping agent behavior and user experience.
- Effective agentic workflows require careful design of planning, memory, and tool use across the stack to support autonomous reasoning and adaptive execution.
- Robust agentic systems depend on scalable infrastructure, clean and governed data, model observability, and well-defined safeguards to ensure reliability and trust.



Q&A

